



Reading

list



Congratulations!

You've completed Course 5:
Add Capabilities with Tools!

This course transformed your agents from intelligent responders into action-capable assistants. You've implemented the **tools** component of the agent equation, giving your agents the power to search the web, execute code, access databases, and perform custom actions in the real world.

Your transformation:

Before Course 5

Agents limited to LLM training data—could only respond with existing knowledge

After Course 5

Agents can take action—search the web in real-time, execute code, query systems, and integrate your business logic

You've completed the core agent equation:
Agent = model + tools + orchestration

What you've accomplished

You started this course with sophisticated agents that had advanced configuration. You're finishing with agents that can:

- **Access real-time information** through Google Search grounding
- **Perform precise calculations** with code execution
- **Connect to external tools** via MCP servers (filesystem, databases, APIs)
- **Execute your business logic** through custom function tools
- **Handle errors gracefully** with strategic error handling
- **Coordinate multiple tools** in complex workflows
- **Delegate to specialists** using agent-as-tool patterns

Skills gained:

- Understanding tools fundamentally (what they are, how agents use them)
- Using production-ready built-in tools (Google Search, Code Execution)
- Connecting to pre-existing MCP servers for ecosystem tools
- Creating custom function tools tailored to your needs
- Designing strategic instructions that guide effective tool usage
- Handling tool errors and coordinating multi-tool workflows
- Using specialized agents as tools for complex reasoning

The journey:

Module 1

Built your first tool-enabled agent + learned state integration preview

Module 2

Used built-in tools for web search and code execution + state preview

Module 3

Connected to external MCP servers for pre-existing tools

Module 4

Created custom tools with comprehensive state integration

Module 5

Combined tools with strategic instructions, error handling, and state coordination

Enhanced with:

- Visual learning aids (Mermaid diagrams in modules showing tool processes, workflows, error handling)
- State integration across all modules connecting Course 4 (State) with Course 5 (Tools)
- Streamlined scope: Removed forward references, simplified verbose sections
- Practical examples: Multi-step workflows, usage tracking, personalization with state



The journey you've taken

Module 1

Understanding Tools (~30 minutes)

You learned what tools are and how they extend agent capabilities beyond LLM knowledge.

Key concepts mastered:

- ✓ Tools extend agents to perform actions beyond text generation
- ✓ The five-step tool process:
Reasoning → Selection → Invocation → Observation → Finalization
- ✓ Three tool types:
Built-in, Custom Function, Agent-as-Tool
- ✓ The **tools** parameter and how LLMs use function metadata
- ✓ Preview:
How tools can read and write session state for personalization

Key Insight:

"Tools are modular code components that execute predefined tasks, enabling agents to access real-time information, affect external systems, and overcome training data limitations."

Module 2

Built-in Tools (~25 minutes)

You learned to use ADK's production-ready built-in tools without writing complex integration code.

Key concepts mastered:

- ✓ Google Search tool provides real-time web information with grounding
- ✓ Code Execution tool enables precise calculations and data processing
- ✓ Grounding connects responses to authoritative sources
- ✓ Policy requirement:
Must display search suggestions
- ✓ Current limitation:
One built-in tool per root agent
- ✓ Preview:
State integration with built-in tools

Key Insight:

"Use built-in tools for common capabilities (search, code execution). Use custom tools for business-specific logic. Built-in tools are maintained and optimized by the ADK team."

Module 3

MCP Tools (~25 minutes)

You learned to connect ADK agents to external MCP servers for pre-existing tool capabilities.

Key concepts mastered:

- ✓ MCP (Model Context Protocol) provides standardized tool server connectivity
- ✓ **McpToolset** connects ADK agents to MCP servers
- ✓ Two connection types:
StdioConnectionParams (local) and **SseConnectionParams** (remote)
- ✓ **tool_filter** controls which tools are exposed to the agent
- ✓ MCP ecosystem provides many pre-built tools (filesystem, databases, APIs)
- ✓ Industry adoption: Anthropic, OpenAI, Google, and The Linux Foundation

Key Insight:

"MCP tools let you leverage the ecosystem—use pre-existing, maintained tool servers instead of building everything yourself."

Module 4

Custom Function Tools (~35 minutes)

You learned to write effective custom function tools with proper signatures, type hints, and docstrings.

Key concepts mastered:

- ✓ Function signatures: Descriptive names, type hints, simple parameters
- ✓ Docstrings guide LLM tool selection (what, when, args, returns)
- ✓ Return dictionaries with status keys for LLM comprehension
- ✓ Multiple tools work together through agent orchestration
- ✓ ADK automatically generates tool schema from function metadata
- ✓ Comprehensive state integration: Multi-step workflows, usage tracking, personalization

Key Insight:

"The function's docstring serves as the tool's description and is sent to the LLM. A comprehensive docstring is crucial for the LLM to understand how to use the tool effectively."

Module 5

Combining Tools Effectively (~30 minutes)

You learned to design strategic instructions that guide agents to use multiple tools effectively.

Key concepts mastered:

- ✓ Instruction patterns:
Tool selection, result handling, sequential workflows
- ✓ Error handling:
Different actions for different error types
- ✓ Agent-as-tool pattern preview
(full coverage in future courses)
- ✓ Best practices for tool coordination and design
- ✓ Strategic instructions guide when, how, and in what order to use tools
- ✓ State coordination with tools:
Multi-step workflows, usage tracking, conditional logic

Key Insight:

"It is crucial to clearly instruct the agent on how to handle different return values that a tool might produce—specify whether the agent should retry, give up, or request additional information."

Real-world applications



Application 1: E-commerce Customer Service

Handle order inquiries, process refunds, escalate complex issues using custom tools with strategic instructions and comprehensive error handling.

Application 2: Research and Analysis

Find current information with Google Search and perform data analysis with Code Execution (separate agents due to one-built-in-tool limitation).

Application 3: Multi-Specialist Support

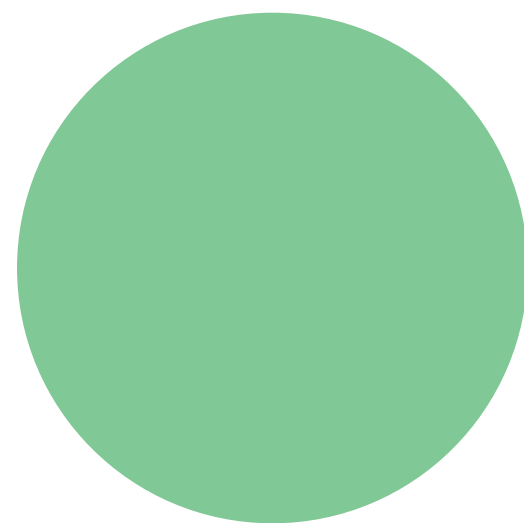
Route customer issues to specialized agents using agent-as-tool pattern with domain-specific expertise.

Application 4: Travel Planning

Search flights and hotels, calculate budgets using multiple custom tools with sequential coordination.

Key takeaways to remember

- 1 The Agent Equation Completed**
Agent = model + tools + orchestration.
Tools enable agents to perform actions beyond text generation.
- 2 Tools Extend LLM Capabilities**
LLMs are limited to training data.
Tools provide real-time information, precise computation, proprietary data access, and external actions.
- 3 LLM Uses Metadata for Tool Selection**
The LLM uses function name, docstring description, and parameter schema to decide which tool to call.
- 4 Built-in vs. MCP vs. Custom Tools Decision**
Use built-in tools for Google capabilities.
Use MCP tools for ecosystem integrations.
Use custom function tools for business-specific logic.
- 5 Status-Based Return Pattern**
Always return dictionaries with status key ('success'/'error'). Include specific error types for different handling.
- 6 Strategic Instructions Guide Tools**
Specify when to use each tool, how to handle results, what order to call tools in, and when to escalate.
- 7 Error Types Need Different Handling**
'not_found' needs clarification,
'invalid_format' needs explanation,
'permission_denied' needs escalation.
- 8 Agent-as-Tool for Specialized Reasoning**
When subtasks require different instructions or expertise, use an agent as a tool instead of a function.



Best practices checklist

✓ Tool Design

- ☐ Descriptive function names (verb-noun pattern)
- ☐ Type hints on all parameters
- ☐ Comprehensive docstrings (what, when, args, returns)
- ☐ Return dictionaries with status key
- ☐ Specific, user-friendly error messages
- ☐ Single, focused purpose per tool
- ☐ JSON-serializable parameter types

✓ Built-in Tools

- ☐ Use Google Search for real-time information
- ☐ Use Code Execution for precise calculations
- ☐ Display search suggestions (policy requirement)
- ☐ Remember one built-in tool per agent limitation
- ☐ Use Gemini 2.0+ models

✓ MCP Tools

- ☐ Check [MCP Registry](#) before writing custom tools
- ☐ Use `tool_filter` to limit exposed tools
- ☐ Use `StdioConnectionParams` for development
- ☐ Use `SseConnectionParams` for production
- ☐ Ensure Node.js is installed for npx-based servers

✓ Instruction Design

- ☐ Reference tools by name
- ☐ Specify when to use each tool
- ☐ Define sequential workflows step-by-step
- ☐ Handle all error types with specific actions
- ☐ Include escalation paths
- ☐ Organize into clear sections

✓ Error Handling

- ☐ Return specific error types
- ☐ Provide human-readable error messages
- ☐ Specify retry vs. give up strategies
- ☐ Define escalation criteria
- ☐ Test all error paths
- ☐ Document expected errors in docstrings



Code patterns reference

Pattern 1: Basic Custom Tool

Python

```
def tool_name(param: type) -> dict:
    """One-line summary. Use when [scenario].

    Args:
        param (type): Description

    Returns:
        dict: {'status': 'success'/'error', ...}
    """
    if validation_fails:
        return {"status": "error", "error_message": "explanation"}
    return {"status": "success", "data": value}
```

Pattern 2: Built-in Tool Usage

Python

```
from google.adk.tools import google_search
from google.adk.code_executors import BuiltInCodeExecutor

# Google Search
agent = LlmAgent(
    model='gemini-2.5-flash',
    tools=[google_search]
)

# Code Execution
agent = LlmAgent(
    model='gemini-2.5-flash',
    code_executor=BuiltInCodeExecutor()
)
```


Pattern 3: MCP Tool Usage

Python

```
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

agent = LlmAgent(
    model='gemini-2.5-flash',
    name='mcp_agent',
    tools=[
        McpToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command='npx',
                    args=['-y', '@modelcontextprotocol/server-filesystem', '/
path'],
                ),
            ),
            tool_filter=['list_directory', 'read_file'], # Security: limit tools
        )
    ],
)
```

Pattern 4: Multi-Tool with Strategic Instructions

Python

```
agent = LlmAgent(
    instruction="""
    ## Tool Selection
    - Use tool_1 when [scenario]
    - Use tool_2 when [scenario]

    ## Workflows
    For [task]:
    1. First use tool_1
    2. Then use tool_2

    ## Error Handling
    - 'not_found': Ask user to verify
    - 'invalid_format': Explain correct format
    """,
    tools=[tool_1, tool_2]
)
```

Pattern 5: Agent-as-Tool

```
Python
from google.adk.tools.agent_tool import AgentTool

specialist = LlmAgent(
    name='specialist',
    instruction="Domain-specific expertise..."
)

main_agent = LlmAgent(
    instruction="Route to specialist when [scenario]",
    tools=[AgentTool(agent=specialist)]
)
```

Common pitfalls to avoid

✗ Tool Design

- Vague function names (`process()` instead of `process_refund()`)
- Missing type hints (LLM needs type information)
- Poor docstrings (LLM relies on these for tool selection)
- Returning strings instead of dicts (structured data is clearer)
- Multi-purpose tools (focus on single purpose)

✗ Instructions

- Too vague (doesn't guide tool usage)
- No error guidance (LLM doesn't know how to handle failures)
- Missing sequences (no order for multi-tool workflows)
- No escalation path (agent gets stuck)

✗ Built-in Tools

- Mixing built-in + custom in same agent (not supported)
- Wrong model version (need Gemini 2.0+)
- Ignoring search suggestion policy (mandatory requirement)

✗ MCP Tools

- Not checking MCP Registry before writing custom tools
- Exposing all tools without `tool_filter` (security risk)
- Missing Node.js prerequisite for npx servers

✗ Error Handling

- Generic error returns (can't differentiate failures)
- No user guidance in errors (help users fix problems)
- Retry loops without changes (same input = same failure)
- Not documenting error types in docstrings

Quick reference card

Tool Types Comparison

Type	Setup	Best For	Maintenance
Built-in	Import from ADK	Common capabilities	ADK team
MCP	Connect to MCP server	Pre-existing ecosystem tools	Server maintainers
Custom Function	Write Python function	Business logic	You
Agent-as-Tool	Create LlmAgent	Complex reasoning	You

When to Use Which

Scenario	Approach	Why
Web search	Built-in: <code>google_search</code>	Production-ready, maintained
Calculations	Built-in: <code>BuiltInCodeExecutor</code>	Safe code execution
Pre-existing capability	MCP tools	Community maintains it
Your business logic	Custom function tool	Your specific needs
Complex reasoning	Agent-as-tool	Requires adaptation

Error Handling Quick Guide

Error Type	Action	Example
<code>not_found</code>	Ask user to verify	"Could you double-check the order ID?"
<code>invalid_format</code>	Explain with example	"Order IDs start with 'ORD' (e.g., ORD123)"
<code>permission_denied</code>	Escalate immediately	"Let me connect you with a supervisor"
<code>temporary_failure</code>	Retry once, then escalate	"Let me try again... Still having issues"

Essential Imports

Python

```
from google.adk.agents import LlmAgent
from google.adk.tools import google_search
from google.adk.code_executors import BuiltInCodeExecutor
from google.adk.tools.agent_tool import AgentTool
from google.adk.tools.mcp_tool import McpToolset
```



Community and support

Primary references

- [Custom Tools Overview](#) - Complete guide to creating custom tools in ADK
- [Built-in Tools](#) - Google Search, Code Execution, and other production-ready tools
- [MCP Tools Integration](#) - Integrating Model Context Protocol tools
- [Function Tools](#) - Detailed documentation on function signatures, docstrings, and patterns
- [Google Search Grounding](#) - Using Google Search tool with grounding and citations

MCP resources

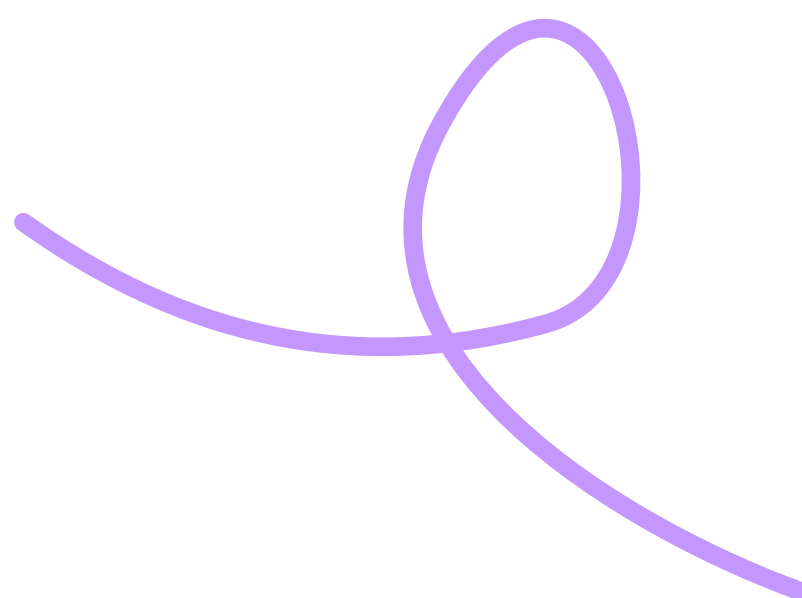
- [MCP Server Registry](#) - Official catalog of MCP servers
- [Official MCP Documentation](#) - Protocol specification and guides
- [MCP GitHub](#) - Reference implementations

❓ Questions? Post questions in the [community forum](#).

Resources

From the community

- [Tools Make an Agent: From Zero to Assistant with ADK](#) - Official Google Cloud Blog providing comprehensive overview of function tools, built-in tools, and tool orchestration patterns
- [Google ADK Masterclass Part 2: Adding Tools to Your Agents](#) - Detailed tutorial on adding tools to agents with hands-on examples and best practices
- [Building AI Agents with Google's ADK: Part 2 — Function Calling](#) - Deep dive into custom function tools covering signatures, docstrings, and return patterns
- [A Guide to ADK Tools: Leveraging Function Tools and Built-in Tools](#) - Practical guide with hands-on examples for both built-in and custom tools
- [Use Google ADK and MCP with an external server](#) - Official guide for integrating MCP (Model Context Protocol) tools for advanced tool integration



What's next?

Course 6: Add Guardrails with Callbacks

Implement monitoring, safety, and control

Final thoughts

You started this course with agents that could think and respond intelligently. You're finishing with agents that can act in the world.

Your transformation is significant

From agents limited by LLM training data to agents that can search the web in real-time, execute code for precise calculations, access your databases, call your APIs, and perform your business logic. This is the difference between a chatbot and an assistant.

The skills you've built are foundational

Strategic instruction design, error handling patterns, tool coordination, and the agent-as-tool pattern—these aren't just techniques for this course. They're core principles you'll apply throughout the ADK Learning Path and in every agent you build.

The core equation is complete

Agent = model + tools + orchestration

You understand the model (Courses 2-3). You've implemented tools (Course 5). Next, you'll master orchestration in future courses.

❓ **Questions or feedback?**
Share in the [ADK Community Forum](#)

Ready for the next challenge?

Future courses will teach you to monitor, control, coordinate, and scale the tool-enabled agents you can now build.